МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Д. М. Мишин
Группа: М8О-307БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Ознакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научиться работать с матрицами перспективы, ортографической проекции и аффинными преобразованиями.

Вариант 18. Сфера.

Метод решения

Сфера строится процедурно в сферических координатах. Точка на сфере радиуса r задаётся углами $\varphi \in [0, \pi]$ и $\theta \in [0, 2\pi]$:

$$x = r \cdot \sin\varphi \cdot \cos\theta,\ y = r \cdot \cos\varphi,\ z = r \cdot \sin\varphi \cdot \sin\theta$$

Диапазоны углов делятся на 8 полос и 10 секторов, что даёт $(8 + 1) \cdot (10 + 1) = 99$ вершин. Каждая ячейка сетки разбивается на два треугольника с одинаковым направлением обхода, всего $8 \cdot 10 \cdot 6 = 480$ индексов. Цвет вершины назначается по её позиции в локальной системе координат отображением из $[-r, r]$ в $[0, 1]$: $c = (p + r) / 2r$. В вершинном шейдере он умножается на цвет, выбранный в интерфейсе.

Вершина переводится из локальных координат в пространство отсечения произведением трёх матриц 4×4 в однородных координатах:

$$p' = P \cdot V \cdot M \cdot p,\ M = T \cdot R \cdot S$$

Матрица модели M составлена из аффинных преобразований: масштабирования S, поворотов R вокруг осей X, Y, Z и переноса T. При таком порядке объект растягивается и вращается вокруг собственного центра, а затем переносится. Матрица вида V задаёт камеру в точке (0, 0, 6), направленную в начало координат. Матрица проекции P переключается в интерфейсе. Перспективная строится по углу обзора, отношению сторон окна, ближней и дальней плоскостям; после неё видеоядро делит координаты на w, поэтому далёкие объекты выглядят меньше. Ортографическая задаёт видимый параллелепипед, и размер объекта от расстояния не зависит. Отношение сторон окна входит в обе матрицы, поэтому сфера не искажается при изменении размеров окна. В Vulkan глубина лежит в диапазоне [0, 1], а ось Y направлена вниз, поэтому матрицы GLM строятся с макросом GLM_FORCE_DEPTH_ZERO_TO_ONE, а элемент P_{22} умножается на -1.

Анимация описывается фазой ψ, которая накапливается по времени: $\psi = \psi + v \cdot \Delta t$, где v – скорость, Δt – время между кадрами. Центр сферы движется по окружности с колебанием по высоте на удвоенной частоте, угол собственного вращения пропорционален фазе:

$$x = R \cdot \cos \psi,\ y = H \cdot \sin 2\psi,\ z = R \cdot \sin \psi,\ \alpha = k \cdot \psi$$

Накопление фазы позволяет ставить анимацию на паузу и менять скорость без скачков положения.

Общие объекты Vulkan: VkInstance, VkPhysicalDevice и VkDevice (подключение к видеоядру), VkQueue (очередь команд), VkSurfaceKHR и VkSwapchainKHR (вывод в окно), VkRenderPass и VkFramebuffer с вложениями цвета и глубины, VkCommandBuffer (команды кадра), VkSemaphore и VkFence (синхронизация), VmaAllocator (память). Для достижения цели работы созданы:

- два VkBuffer с памятью VmaAllocation: вершинный (99 вершин по 24 байта: позиция и цвет) и индексный (480 индексов uint32_t),
- два VkShaderModule: вершинный и фрагментный шейдеры на GLSL, скомпилированные в SPIR-V,
- VkPipelineLayout с диапазоном push-констант (VkPushConstantRange) размером 80 байт: матрица MVP и цвет-множитель,
- VkPipeline: список треугольников, описание формата вершины, отбрасывание задних граней, тест глубины (VK_COMPARE_OP_LESS), динамические область вывода и прямоугольник отсечения.

Каждый кадр в буфер команд записываются vkCmdBeginRenderPass, vkCmdBindPipeline, vkCmdPushConstants, vkCmdSetViewport, vkCmdSetScissor, vkCmdBindVertexBuffers, vkCmdBindIndexBuffer, vkCmdDrawIndexed и vkCmdEndRenderPass. Данные сферы загружаются в буферы один раз, а всё движение достигается изменением матрицы, передаваемой через push-константы. Параметры проекции, преобразований, цвета и анимации задаются элементами интерфейса ImGui.

Результаты

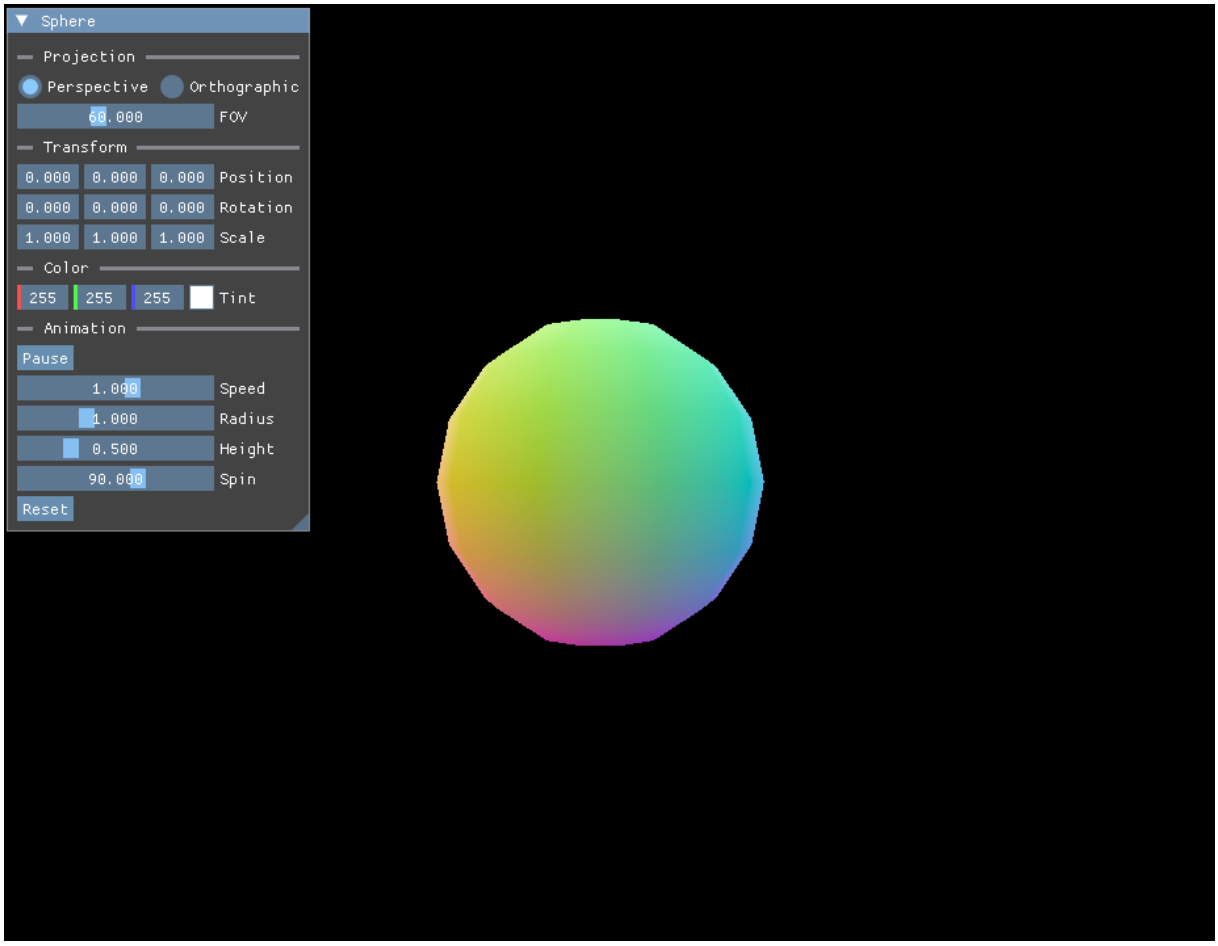


Рис. 1. Начальное состояние сферы.

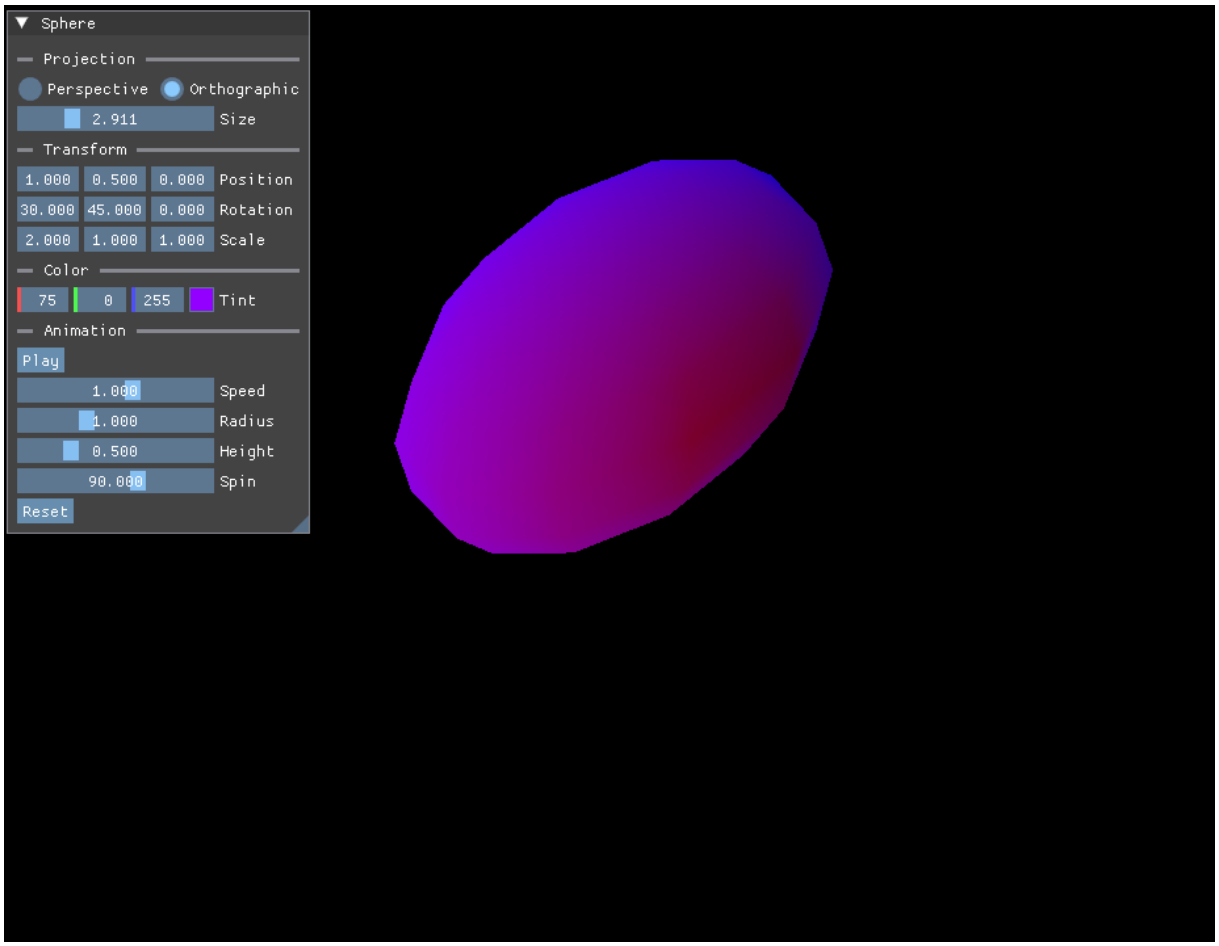


Рис. 2. Измененные параметры.

Выводы

Я научился процедурно строить трёхмерный объект: сетку сферы в сферических координатах с вершинным и индексным буферами. Разобрался, как устроен графический конвейер Vulkan: создал шейдеры, pipeline с тестом глубины и отбрасыванием задних граней, передал данные в шейдер через push-константы. Освоил аффинные преобразования и матрицы модели, вида и проекции, на практике увидел разницу между перспективной и ортографической проекциями и реализовал интерфейс на ImGui для управления сценой и анимацией.